CANNBot 开发进阶：TileLang-Ascend 算子开发实操

作者：杨犇

时间：2026-4-16

https://gitcode.com/cann/skills

CANN

目录

Part 1 TileLang-Ascend 算子开发模式

Part 2 TileLang-Ascend Agent 技能介绍

Part 3 Softmax 算子生成实例

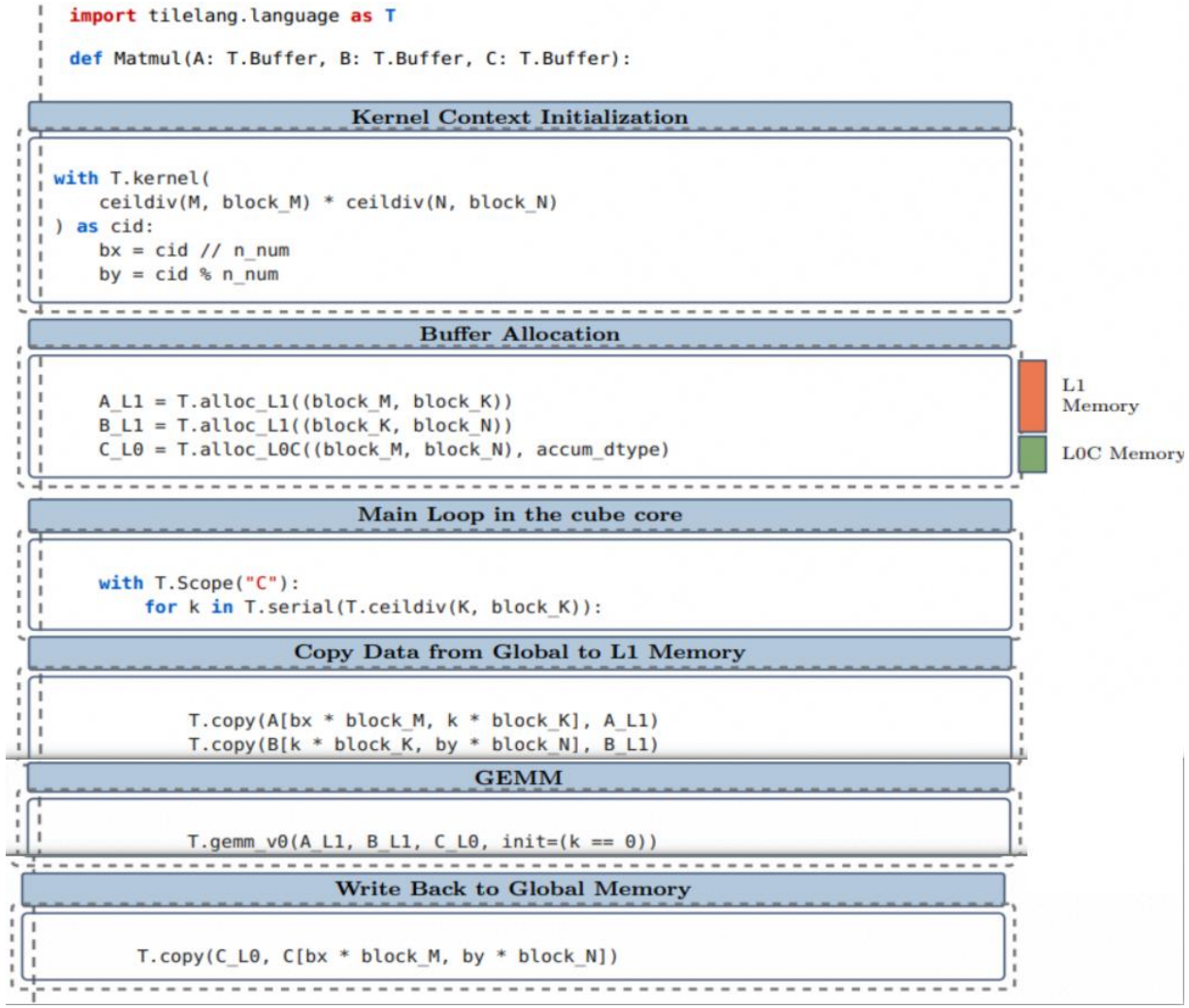
TileLang-Ascend 算子开发模式

TileLang

- Python化tile粒度算子编程DSL
- 核心技术团队来自北京大学
- 2025-1-20 开源

TileLang-Ascend

- TileLang在昇腾平台适配。
- 2025-9-29 开源



TileLang-Ascend 算子开发模式

Developer

```python
with T.Kernel(m_num * n_num, is_npu=True) as (cid, _):
    bx = cid // n_num
    by = cid % n_num

    A_L1 = T.alloc_shared((block_M, K_L1), dtype)
    B_L1 = T.alloc_shared((K_L1, block_N), dtype)
    C_L0 = T.alloc_fragment((block_M, block_N), accum_dtype)

    loop_k = T.ceildiv(K, K_L1)
    for k in T.serial(loop_k):
        T.copy(A[bx * block_M, k * K_L1], A_L1)
        T.copy(B[k * K_L1, by * block_N], B_L1)
        T.gemm_v0(A_L1, B_L1, C_L0, init=(k == 0))
    T.copy(C_L0, C[bx * block_M, by * block_N])
```

Managed Tile Programming

Expert

```python
for kk in T.serial(loop_kk):
    if kk == 0:
        T.wait_flag("mte2", "mte1", k % S1)
    T.wait_flag("m", "mte1", kk % S2)
    T.copy(A_L1[k % S1, 0, kk * block_K], A_L0[kk % S2, :, :])
    T.copy(B_L1[k % S1, kk * block_K, 0], B_L0[kk % S2, :, :])
    if kk == 3:
        T.set_flag("mte1", "mte2", k % S1)
    T.set_flag("mte1", "m", kk % S2)
    T.wait_flag("mte1", "m", kk % S2)
    T.mma(A_L0[kk % S2, :, :], B_L0[kk % S2, :, :], C_L0,
          init=T.And(k == 0, kk == 0))
    T.set_flag("m", "mte1", kk % S2)
```

Explicit Tile Programming

CANN

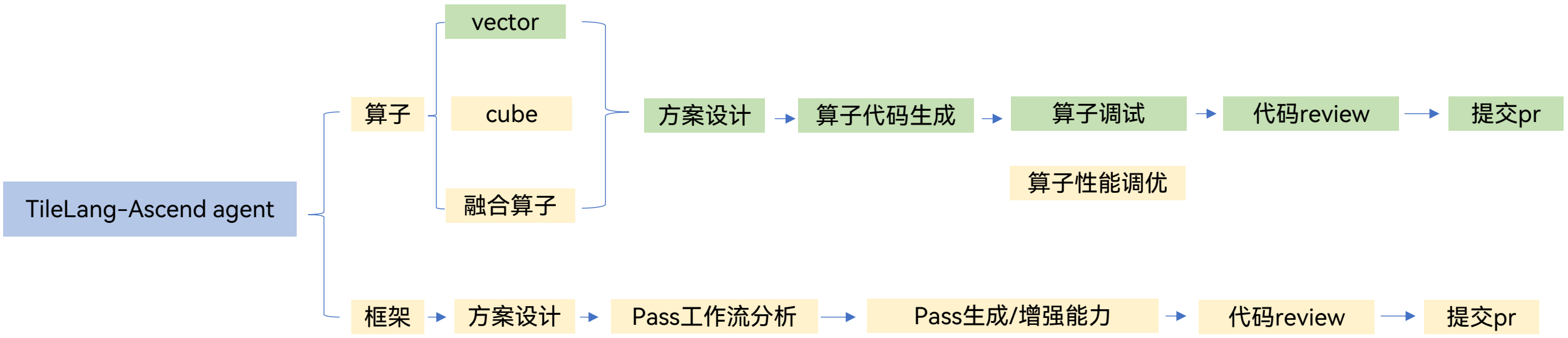
目录

Part 1 TileLang-Ascend 算子开发模式

Part 2 TileLang-Ascend Agent 技能介绍

Part 3 Softmax 算子生成实例

TileLang-Ascend Agent 建设规划 & 关键技能介绍



算子方案设计skill

目标：根据算子需求，生成完成算子方案设计文档，指导下一步算子开发使用。

算子信息输入

↓

解析算子特征

↓

伪代码与api映射

↓

验证方案

↓

质量自检

↓

输出设计方案

name	tilelang-op-design
description	根据算子需求生成 TileLang-Ascend 算子设计文档（design.md）。涵盖编程模式选型（Developer/Expert/混合）、API 映射、内存层级规划、Tiling 策略、循环结构、同步策略、验证方案等。触发：设计算子、生成 design.md、算子方案设计、新算子开发、算子实现方案。

TileLang-Ascend 算子设计文档生成

1. 目标

根据算子需求信息，生成一份完整的 TileLang-Ascend 算子设计文档（`design.md`），涵盖以下核心决策：

- **编程模式选型**：Developer / Expert / 混合模式
- **API 映射**：将数学公式拆解为 TileLang DSL 原语组合
- **内存层级规划**：GM → L1/UB → L0 的数据搬运路径
- **Tiling 策略**：Block 划分与 Tile Shape 设计
- **循环结构**：T.Parallel / T.serial / T.Pipelined / T.Persistent 的选择
- **同步策略**：自动同步 vs 手动同步标志
- **验证方案**：Golden 函数与多级测试计划

2. 输入要求

必需信息

字段	说明
算子名称	如 `softmax`、`layer_norm`、`flash_attention`
数学公式	算子的数学表达，如 $\text{softmax}(x_i) = e^{x_i} / \sum e^{x_j}$
输入张量规格	shape、dtype
输出张量规格	shape、dtype
编程模式偏好	Developer / Expert / 混合

CANN

算子代码生成skill

目标：根据算子设计文档，生成算子代码与测试用例，确保精度和功能正确。

提取方案设计关键信息

↓

根据api实现和库上用例生成算子代码

↓

运行验证与错误调试

↓

精度与功能正确

↓

任务完成

name	tilelang-op-generate
description	基于设计文档生成 TileLang-Ascend 算子实现代码与测试。从 design.md 中提取关键信息，结合 examples/ 中的参考实现生成可运行代码。触发：实现算子、写 kernel、生成代码、算子编码、根据设计文档实现。

TileLang-Ascend 算子代码生成

基于设计文档（`design.md`）和已有示例，生成可运行的算子实现与测试。

---

1. 从 design.md 中提取的信息（只取这些）

design.md 可能很长，**只提取以下字段，忽略其余内容**：

提取字段	所在章节	用途
数学公式	§1 概述	理解计算逻辑
算法步骤分解	§1 算法描述	确定计算顺序
API 映射表	§3 API 映射设计	**核心**：每步用哪个 TileLang API
伪代码	§3 计算伪代码	**核心**：代码骨架
输入输出 shape 和 dtype	§4 数据规格	函数签名和测试数据
block 大小	§5 Tiling 策略	分块参数
pass_configs	§7 同步策略	JIT 配置
Golden 函数	§8 验证方案	测试对比基准
测试用例表	§8 验证方案	测试配置
精度标准	§8 验证方案	atol / rtol

CANN

代码review skill

目标：对编写的代码进行review，检查格式问题和潜在bug，并自动修复。

确认review范围

↓

环境检查(ruff，clang-format)

↓

运行review、生成报告

↓

询问修复

↓

执行修复

name	tilelang-review-skill
description	检查代码格式是否符合 CI 规则。自动检测并安装缺失工具（ruff、clang-format），先运行检查生成报告，使用醒目方式询问用户后，仅在用户同意时执行修复。工作流程：检测环境→自动安装缺失工具→运行检查→生成报告→醒目询问→用户确认→执行修复。使用此技能当：用户要求"格式检查"、"格式化代码"、"代码格式化"、"检查代码格式"、"代码 review"、"代码审查"、"修复格式"、"fix format"、"lint code"、"检查代码规范"、提交 PR 前验证、或需要检查 Python/C++ 代码风格时。

代码格式检查

检查代码格式并生成详细报告，与 CI 规则保持一致。

⚠️ 核心原则

询问范围 + 自动安装 + 检查和修复分离： 0. **询问范围**: 首先询问用户检查范围（修改文件 / 全部文件）

1. **自动安装**: 检测缺失工具并自动安装（ruff、clang-format）
2. **检查阶段**: 只运行检查脚本，生成报告，**不修改任何文件**
3. **询问阶段**: 使用醒目的方式询问用户是否修复
4. **修复阶段**: 只有用户明确同意后才运行修复脚本

```
🚫 检查时严禁自动修复！必须等待用户确认！
```

CANN

目录

Part 1 TileLang Ascend介绍

Part 2 TileLang Agent技能介绍

Part 3 Softmax 算子生成实例

Softmax 算子生成实例

Softmax算子在LLM模型中的应用

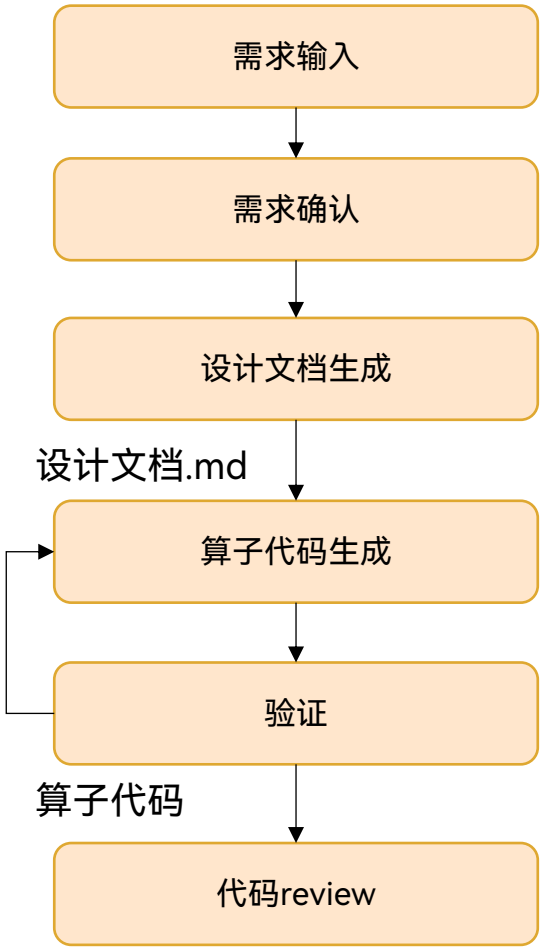
- Attention：Softmax 将相似度转为权重(归一化注意力)
- 输出层：Softmax 将 logits 转为下一个 token 的概率分布
- 训练：Softmax + 交叉熵用于计算损失

数学表达式

$$m = max(x)$$

$$\mathrm{Softmax}(x_i) = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}$$

开发流程预览



CANN

Softmax 算子生成实例

1、需求输入

opencode

请生成一个softmax算子，用tilelang-ascend实现

Build GLM-5 Model Studio Coding Plan

tab agents ctrl+p commands

• Tip OpenCode auto-formats files using prettier, gofmt, ruff, and more

请生成一个softmax算子，用tilelang-ascend实现

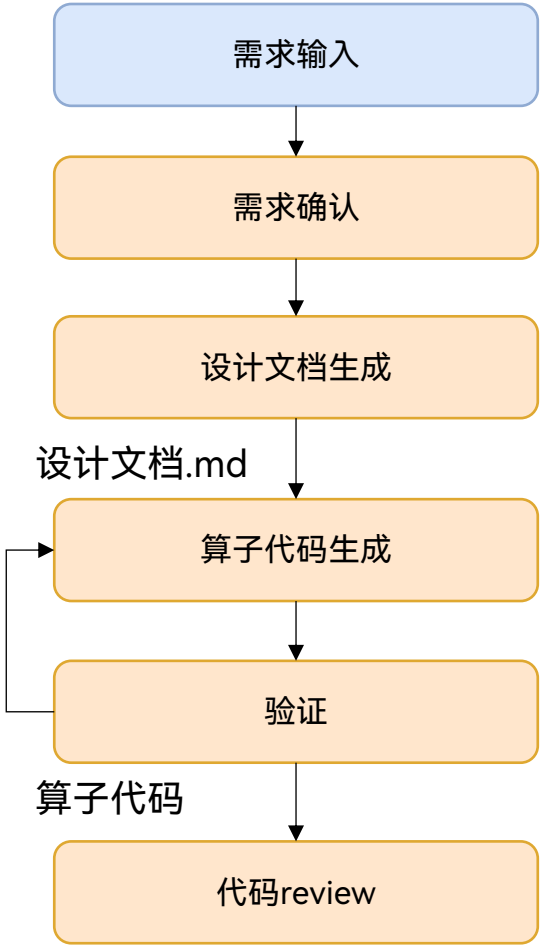
Thinking: 用户要求生成一个 softmax 算子，使用 tilelang-ascend 实现。

根据 AGENTS.md 的指导，我需要：
1. 先了解 softmax 的需求
2. 选择编程模式（Developer 或 Expert 模式）
3. 参考 examples/ 中的类似实现
4. 创建独立的 examples/softmax/ 目录

首先，我应该询问用户想使用哪种编程模式，这是 AGENTS.md 中明确要求的：
"必须询问使用 Developer 模式、Expert 模式或混合编程模式，否则不进入下一步"

让我先加载 tilelang-op-design skill 来帮助设计这个算子。

我需要先了解您的需求。让我加载算子设计 skill 来帮您完成这个任务。



CANN

Softmax 算子生成实例

2、需求确认

根据 skill 的要求：
1. 每次只询问一个字段
2. 按表格顺序依次询问：算子名称 → 数学公式 → 输入张量规格 → 输出张量规格 → 编程模式偏好
3. 已提供的字段跳过

用户已经提供了：
- 算子名称：softmax（从请求中可以推断）

接下来我需要询问数学公式。

Questions

请提供 softmax 算子的数学公式（可选择标准 softmax 或自定义）
标准 softmax

请提供 softmax 算子的数学公式（可选择标准 softmax 或自定义）

1. 标准 softmax
   对最后一维计算 softmax：y_i = exp(x_i - max(x)) / sum(exp(x_j - max(x)))
2. FlashAttention softmax
   处理 attention score，支持 scaled dot-product attention
3. 多维 softmax
   支持指定轴计算的 softmax
4. Type your own answer

↑↓ select enter submit esc dismiss

Thinking: 用户选择了标准 softmax。接下来我需要询问输入张量规格。

Questions

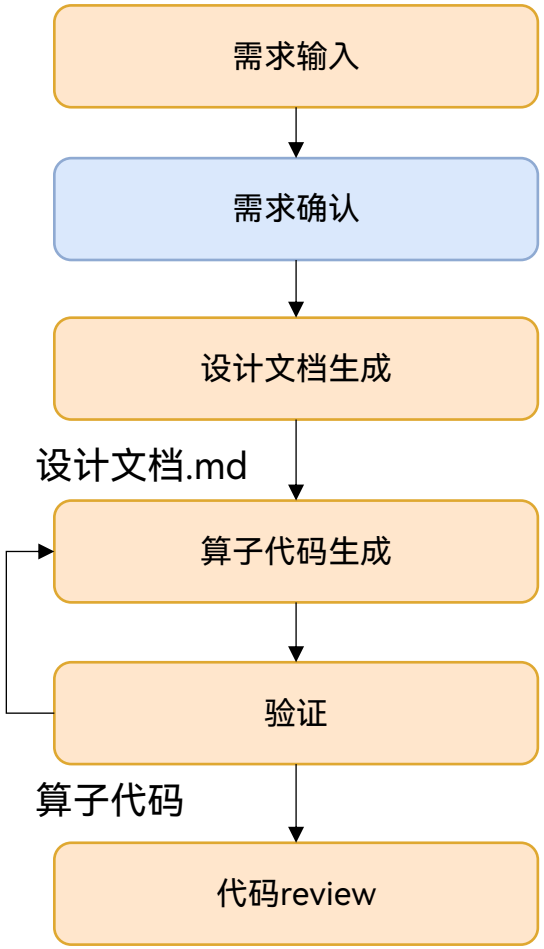
请提供输入张量的 shape 和 dtype
(M, N), float16

Thinking: 用户选择了 (M, N), float16。接下来我需要询问输出张量规格。

Questions

请提供输出张量的 shape 和 dtype
(M, N), float16 (Recommended)

Thinking: 用户选择了 (M, N), float16。接下来我需要询问编程模式偏好。



Softmax 算子生成实例

3、设计文档生成

现在我需要输出完成报告，然后询问用户是否要根据此报告生成对应算子代码。

设计文档生成报告

- 算子：softmax
- 编程模式：Developer 模式
- 计算类型：纯 Vector
- 输出路径：examples/softmax/design.md

自检结果

1. 编程模式选型：✅
2. API 映射具体性：✅
3. 内存搬运完整性：✅
4. Tiling 约束分析：✅
5. 同步策略匹配：✅
6. 验证方案覆盖：✅
7. 无占位符：✅

设计要点

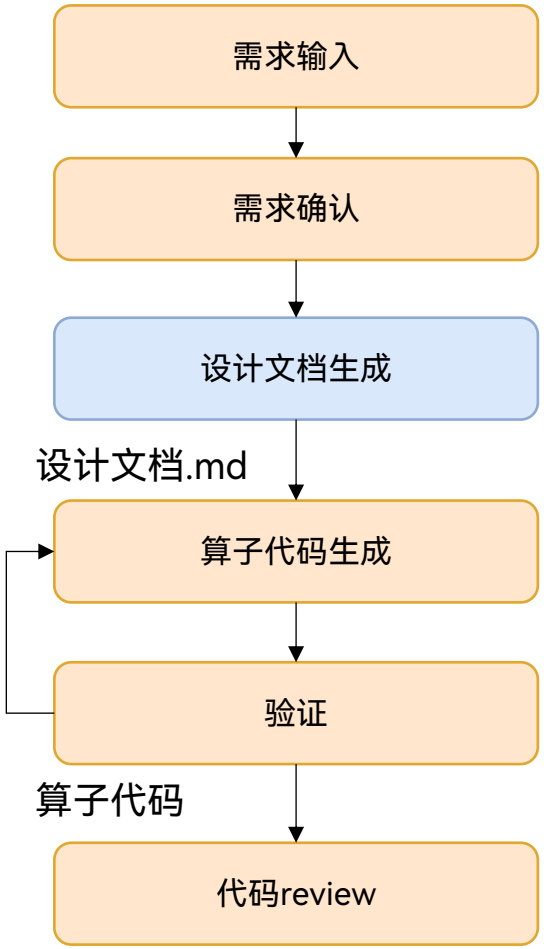
- 使用 float32 计算精度（提高稳定性）
- block_M=64, block_N=128（平衡并行度和 UB 占用）
- 自动同步模式（Developer 模式特性）
- 复用 buffer 降低内存占用

是否要根据此设计文档生成 softmax 算子代码？

▣ Build · GLM-5 · 6m 25s

根据设计文档生成代码吧

Build GLM-5 Model Studio Coding Plan



CANN

Softmax 算子生成实例

4、算子代码生成 & 验证

Softmax 算子生成完成

已成功生成 softmax 算子实现并通过测试验证。

生成的文件

```
examples/softmax/
├── softmax.py        # 算子实现 + 测试（已验证）
├── design.md         # 设计文档（已更新约束）
└── README.md         # 使用说明
```

测试结果

✅ 所有测试通过：
- M=32, N=32, block_M=32, block_N=32
- M=128, N=128, block_M=64, block_N=128
- M=256, N=256, block_M=64, block_N=256
- M=512, N=128, block_M=64, block_N=128

实现特点

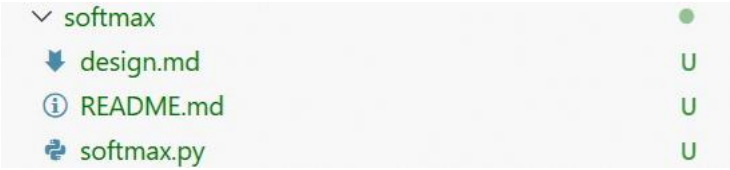
- **编程模式**：Developer 模式 + 自动同步
- **计算精度**：float32 计算 + float16 输入输出
- **Tiling 策略**：M 方向分块（block_M=64），N 方向完整处理
- **VEC_NUM=2**：双核并行处理

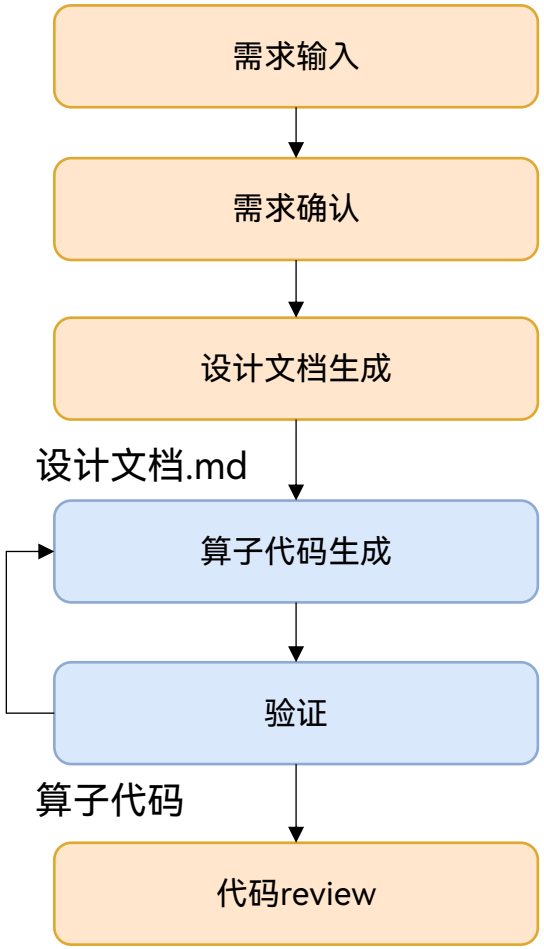
当前限制

⚠️ 实现有以下限制：
1. **M 必须能被 block_M 整除**（不支持尾块）
2. **N 必须等于 block_N**（不支持 N 分块）
3. **block_N ≤ 256**（受 UB 内存限制）

如需支持更复杂的场景（非整除、大 N 分块），需要实现 online softmax 算法或尾块处理逻辑。

Build · GLM-5 · 11m 14s





Softmax 算子生成实例

5、代码review

请review下代码吧

Thinking: 用户要求review代码。我应该使用tilelang-review-skill来检查代码格式是否符合CI规则。

→ Skill "tilelang-review-skill"

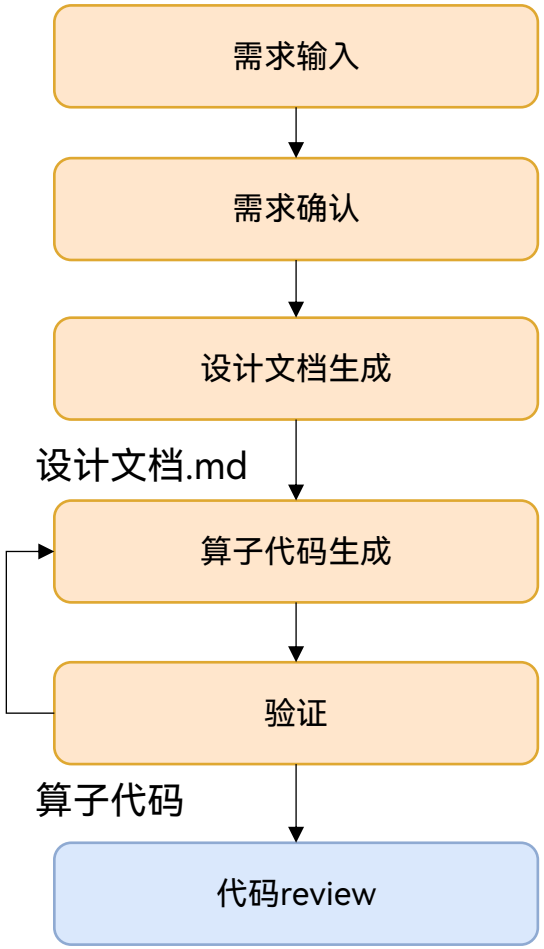
Thinking: 好的，现在我需要按照 tilelang-review-skill 的流程来review代码。

根据skill的要求，我需要：
1. 步骤 0：询问检查范围（修改文件 / 全部文件）
2. 步骤 1：环境检测
3. 步骤 2：运行检查脚本
4. 步骤 3：生成报告
5. 步骤 4：询问修复
6. 步骤 5：执行修复（如果用户同意）
7. 步骤 6：更新报告
8. 步骤 7：检查报告是否生成

✅ 代码格式 Review 完成

修复结果：
- ✅ Python lint 问题已修复（SIM300 - Yoda condition）
- ✅ C++ 文件已格式化
- ✅ 所有文件验证通过

📄 格式检查报告：.agents/reports/format-report-20260413_144035.md



CANN

CANNBot TileLang-Ascend ——让算子开发更简单

开始您的CANNBot TileLang之旅！

欢迎关注、使用、贡献，共建社区！

- https://github.com/tile-ai/tilelang-ascend
- https://gitcode.com/cann/skills

群聊：CANNBot与Skills开源交流群

该二维码7天内(4月21日前)有效，重新进入将更新

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann/skills